

In **.NET 6+**, the difference between `CreateScope()` and `CreateAsyncScope()` in C# is mainly about **whether the scope supports asynchronous disposal**.

1. `CreateScope()`

- **Namespace:** `Microsoft.Extensions.DependencyInjection`
- **Returns:** `IServiceScope`
- **Disposal:** Synchronous (`Dispose()`).
- **When to use:**
Use when your scoped services **do not** implement `IAsyncDisposable` (or you don't care about async disposal).

Example:

```
using Microsoft.Extensions.DependencyInjection;

var serviceProvider = new ServiceCollection()
    .AddScoped<MyService>()
    .BuildServiceProvider();

using (var scope = serviceProvider.CreateScope())
{
    var service = scope.ServiceProvider.GetRequiredService<MyService>();
    service.DoWork();
} // Dispose() called here
```

2. `CreateAsyncScope()`

- **Namespace:** `Microsoft.Extensions.DependencyInjection`
- **Returns:** `AsyncServiceScope` (struct implementing `IAsyncDisposable`).
- **Disposal:** Asynchronous (`DisposeAsync()`).
- **When to use:**
Use when **any scoped service implements** `IAsyncDisposable` (e.g., services that use async cleanup like database connections, streams, etc.).

Example:

```

using Microsoft.Extensions.DependencyInjection;

var serviceProvider = new ServiceCollection()
    .AddScoped<MyAsyncService>()
    .BuildServiceProvider();

await using (var scope = serviceProvider.CreateAsyncScope())
{
    var service =
scope.ServiceProvider.GetRequiredService<MyAsyncService>();
    await service.DoWorkAsync();
} // DisposeAsync() called here

```

Key Differences Table

Feature	CreateScope()	CreateAsyncScope()
Return type	IServiceScope	AsyncServiceScope
Disposal method	Dispose()	DisposeAsync()
Async cleanup support	✗ No	✓ Yes
Introduced in .NET	.NET Core 2.0	.NET 6

When to choose which

- **If all scoped services are synchronous** → CreateScope()
- **If any scoped service implements IAsyncDisposable** → CreateAsyncScope() (to ensure proper async cleanup)

 **Tip:** If you call Dispose() on a scope containing IAsyncDisposable services, their async cleanup **won't run** — which can cause resource leaks.

So in modern .NET apps, **prefer** CreateAsyncScope() unless you are certain no async disposal is needed.

If you want, I can give you a **benchmark example** showing the performance difference between CreateScope() and CreateAsyncScope() in .NET 8.

Would you like me to prepare that?